Engineering Certified JSON Derivers for Rocq with ELPI

Will Thomas and Perry Alexander

University of Kansas, Lawrence, USA
{30wthomas, palexand}@ku.edu

Abstract. Practical verification produces extracted code, and extracted code eventually crosses system boundaries. JSON is a common boundary format for configuration, test data, counterexamples, traces, and tool pipelines, but serializers at this boundary are often hand-written and unverified. We present an ELPI-based deriver for JSON that automatically generates `Jsonifiable` instances for a practical fragment of Rocq inductive and record types. Each generated instance contains an encoder, a decoder, and a Rocq-checked proof that decoding the encoded value recovers the original. The contribution is a performance-conscious tool for extending the verified trust perimeter of extracted Rocq components to their JSON boundary.

1 Introduction

Automated verification is often evaluated by the proof it produces, but practical formal methods also has to run. A verified component may be extracted to OCaml, linked into a test harness, used as a runtime monitor, embedded in a larger tool chain, or deployed as part of a verified system. As soon as that component communicates with the outside world, the proof assistant's internal data representation crosses a boundary.

JSON is a common format at that boundary. Model-checker front-ends exchange structured system descriptions and counterexamples, verified runtime monitors consume event logs, test generators exchange inputs and results, and verified protocols serialize messages for unverified peers. The problem is that the serialization layer is often outside the trusted development. Extraction preserves the behavior of Rocq [3] terms, but it does not automatically certify the hand-written encoder or decoder that maps those terms to a deployment format. A bug in that code can invalidate the deployed verification story.

`rocq-json` addresses this gap by extending the formal trust perimeter to the JSON boundary. It provides the following typeclass:

```
Class Jsonifiable (A : Type) := {
  to_JSON    : A → JSON;
  from_JSON  : JSON → Result A string;
  canonical_jsonification : forall (a : A), from_JSON (to_JSON a) = res a
}.
```

The deriver described here generates all three components for many ordinary inductive and record types. It is implemented in `coq-elpi` and exposed through the command `Elpi derive.jsonifiable T` [2]. For verification-oriented applications that produce JSON, the central guarantee is exactly the displayed one: a value that leaves the verified component through the encoder can be decoded back to the same Rocq value.

This is a tool paper about practical verification engineering with code that may cross assurance boundaries. The substantially new contribution is the integration of ELPI inspection of Rocq declarations, generated code that remains suitable for extraction, typeclass parameters for nested serializable fields, automatic proof construction for the round-trip theorem, and an artifact that measures both applicability and extracted performance. The same ingredients are broadly useful for other derivers that construct computational content and accompanying correctness proofs.

2 Tool Overview

The user-facing interface is deliberately small. Given a declaration such as an enumeration, algebraic data type, parameterized container, or record, the user invokes the deriver and receives a registered typeclass instance. Nullary constructors are encoded as JSON strings. Constructors with arguments are encoded as singleton JSON objects whose key is the constructor name and whose value stores named fields.[1] Records are encoded as JSON objects keyed by projection names.

For example, the following declaration and command are enough to produce an encoder, decoder, proof of *invertibility*, and registered typeclass instance:

```
Inductive UserRole : Type :=
| Admin
| Moderator (level : nat)
| StandardUser (id : nat) (username : string)
| Guest.
Elpi derive.jsonifiable UserRole.
```

The generated representation uses a string for nullary constructors and an object for constructors with fields:

```
Example role_roundtrip (r : UserRole) :
  from_JSON (to_JSON r) = res r := canonical_jsonification r.

Example moderator_json :
  to_JSON (Moderator 2) =
    JSON_Object [("Moderator", JSON_Object [("level", JSON_Nat 2)])] :=
  eq_refl.
```

[1] For anonymous arguments, a suitable name is inferred from the *type* of the argument

For parameterized types, the generated instance takes serializers for type parameters. For recursive types, the generated encoder and decoder use structurally recursive definitions. The current implementation also supports a conservative collection of nested recursive occurrences through standard data containers such as lists, options, and products. More general nested or mutually recursive types are currently reported as unsupported rather than handled partially.[2]

```
Inductive BinTree (A : Type) : Type :=
| BinLeaf
| BinNode (left : BinTree A) (value : A) (right : BinTree A).
Elpi derive.jsonifiable BinTree.
Check BinTree_Jsonifiable :
  forall A, Jsonifiable A → Jsonifiable (BinTree A).
```

The generated proof is part of the public contract. A successful derivation does not merely produce functions with plausible behavior; it registers a `canonical_jsonification` proof. This property is intentionally one-sided. It says that values produced by the encoder can be decoded back to the original value. For extracted verified components that emit JSON for other tools or services, this is the direction that protects the boundary: the serialized message has not lost or changed the Rocq value. The converse property, that every accepted JSON value is a canonical representation, is a different contract. It would require specifying a canonical JSON grammar for each type and proving that the decoder rejects all non-canonical alternatives, which is useful but substantially more representation-specific.

3 Implementation

The deriver follows the shape of the Rocq declaration. ELPI is an embedded λProlog language used by `coq-elpi` to inspect and construct Rocq terms at the meta-level. The deriver first reads the inductive or record view, classifies parameters and constructor arguments, and then builds separate skeletons for the encoder, decoder, proof, and final typeclass instance. The skeletons are elaborated by Rocq before being added as constants. This staging has been useful in practice: ELPI performs structural analysis and code generation, while Rocq remains responsible for checking the terms that become part of the trusted development.

Two engineering choices are central. First, field serializers are kept abstract through typeclass arguments where possible. This makes generated instances compose with existing hand-written or derived instances, including standard containers. In practice, this means that although `nat` in Rocq is inductively defined as `O | S _`, we are better served by making it an actual number (e.g. 0, 1, 42). Second, decoder generation is performance-sensitive. Large enumerations

[2] The issue of generating quality `to/from_JSON` functions is not as difficult as proving them correct without quality structural recursion procedures. This may be mitigated as better support for nested inductives in Rocq is actively being pursued [1]

are decoded using a string decision tree that extracts to direct pattern matching over characters, avoiding a long chain of string equality tests. The resulting extracted code is close to the shape a careful and performance oriented user would write by hand.

The proof-generation strategy is mixed. General recursive and record cases use Rocq proof automation specialized to the generated encoders and decoders. Pure enumerations use a direct generated proof term, avoiding proof-search overhead on a common stress case. This division is not theoretically deep, but it matters for predictable tool latency.

4 Evaluation

We evaluate two questions: how broadly the deriver applies to existing Rocq declarations, and whether generated extracted code is competitive with hand-written code on a stress case.

4.1 Applicability

The artifact includes a benchmark script that scans installed Corelib and Stdlib sources for inductive-like declarations, probes each candidate in isolation, records diagnostics, and then compiles a single benchmark file containing all successful derivations. This workflow is deliberately conservative: unsupported declarations remain visible as categorized failures.

Table 1. Corelib/Stdlib applicability summary.

Metric	Count
Discovered declarations	343
Probed declarations	343
Successful probes	78
Probe failures	265
Probe timeouts	0
Timed successful derivations	78

The successful probes include 47 inductives, 17 variants, and 14 records or structures; 39 are from Corelib and 39 are from Stdlib.

The category names in this table are not meant to declare every failure "invalid input". They are a triage device. Some categories are semantic non-targets for this version of the tool, such as propositions or coinductive streams. Others are engineering limitations or possible tool failures that should be treated as future work. Table 3 gives one representative declaration for each category and the rationale used by the artifact.

Table 2. Failure categories reported by the probe benchmark.

Category	Count
prop-target-not-supported	167
dependent-field-not-supported	48
indexed-family-not-supported	21
scanner-or-logical-path	18
function-field-not-supported	4
proof-field-not-supported	3
sort-field-not-supported	3
coinductive-not-supported	1

Table 3. Representative failure examples and classification rationale.

Category and example	Rationale
`prop-target-not-supported: or`	The target lives in `Prop`, so ordinary elimination into JSON data in `Type` is not available.
`dependent-field-not-supported: sig`	A later constructor field depends on an earlier witness; plain JSON does not reconstruct dependency.
`indexed-family-not-supported: reflect`	The declaration is indexed by values rather than being a plain data type after parameters.
`scanner-or-logical-path: FMapAVL.Raw.tree`	The source scanner guessed a logical name that is not directly derivable.
`function-field-not-supported: implies`	Arbitrary functions have no generic JSON representation.
`proof-field-not-supported: sumbool`	Constructor fields contain proof evidence that would need erasure or reconstruction.
`sort-field-not-supported: Tlist`	Constructor fields contain sorts such as `Type`; the deriver serializes values, not universes.
`coinductive-not-supported: Stream`	Infinite data requires a different fuel-bounded or lazy serialization contract.

The important distinction is between unsupported-by-design and unresolved. The paper claims the former only for cases where the current `Jsonifiable` interface has no clear computational target, for example `Prop` targets or potentially infinite coinductive data. The remaining unsupported categories name the blocking shape directly: dependent fields, indexed families, proof/SProp fields, sort-valued fields, or function fields. The `scanner-or-logical-path` category is a limitation of the benchmark harness. All categories are reported explicitly so the success rate is auditable rather than inflated by silent filtering.

The raw success rate is therefore intentionally not the main applicability claim. Of the 343 discovered declarations, 167 live in `Prop`, one is coinductive, seven contain sort-valued or function-valued fields, and 18 are scanner or logical-path misses rather than deriver inputs. Removing just these semantic non-targets and harness misses leaves 150 computational declarations that the benchmark could plausibly ask this tool to handle; 78 of them derive, or 52.0%. The remaining cases are visible limitations, chiefly dependent fields and indexed families, not hidden failures.

4.2 Derivation Time

The combined successful benchmark records Rocq's `Time` output for each derived instance. In the generated run used for this draft, the combined benchmark compiled successfully with wall-clock time 5.12 seconds; the sum of Rocq-reported derivation transactions was 4.35 seconds. The mean derivation time was 0.056 seconds, the median was 0.035 seconds, the 90th percentile was 0.111 seconds, and the maximum was 0.564 seconds.

Table 4. Slowest successful derivations in the broad benchmark.

Declaration	Kind	Seconds
Corelib.Init.Byte.byte	Inductive	0.564
Stdlib.btauto.Reflect.formula	Inductive	0.180
Stdlib.rtauto.Rtauto.proof	Inductive	0.176
Stdlib.micromega.ZMicromega.ZArithProof	Inductive	0.163
Corelib.Floats.FloatClass.float_class	Variant	0.155
Stdlib.micromega.RingMicromega.Psatz	Inductive	0.153
Stdlib.Sorting.Heap.Tree	Inductive	0.145
Stdlib.setoid_ring.Field_theory.FExpr	Inductive	0.113

The slowest success is `byte`, a 256-constructor Corelib enumeration. It is the broad benchmark's version of the large enum stress case used while engineering the string decision tree. The remaining cost is dominated by Rocq-side derivation and proof construction; after extraction, the corresponding dispatch path is close to the handwritten Rocq baseline below.

4.3 Extracted Code Performance

The extraction benchmark compares generated definitions against handwritten Rocq definitions after both have been extracted to OCaml. This keeps the comparison in the same source language and extraction pipeline. The benchmark covers several shapes: a 256-constructor enumeration, a tuple-like constructor, a flat record, two mixed sums with nullary and field-bearing constructors, and a recursive binary tree. Each row reports `JSONIFIABLE_EXTRACTION_RUNS` samples, which defaults to 10. The enum benchmark serializes and deserializes every constructor; the other benchmarks run many iterations over representative values. These benchmarks are still small, but they exercise different generated-code paths and are more informative than a single stress case.

Table 5. Extracted round-trip benchmarks.

Benchmark	Samples	Mean	Median	Min	Max
enum256_generated	10	0.042716	0.041911	0.040905	0.047295
enum256_handwritten	10	0.042939	0.042664	0.041016	0.049128
point2d_generated	10	0.001006	0.000966	0.000877	0.001302
point2d_handwritten	10	0.001134	0.001093	0.000995	0.001508
userrole_generated	10	0.001736	0.001708	0.001628	0.001995
userrole_handwritten	10	0.002499	0.002430	0.002377	0.002733
serverconfig_generated	10	0.000618	0.000589	0.000567	0.000761
serverconfig_handwritten	10	0.001475	0.001443	0.001374	0.001666
instruction_generated	10	0.001680	0.001678	0.001579	0.001799
instruction_handwritten	10	0.002574	0.002548	0.002487	0.002689
bintree_generated	10	0.003335	0.003302	0.003010	0.003749
bintree_handwritten	10	0.004969	0.004976	0.004527	0.005391

The intended conclusion is not that these microbenchmarks predict every serializer workload. Rather, they check that the deriver does not impose an obvious extraction penalty on common shapes and on a case that stresses constructor dispatch.

5 Related Work and Positioning

Rocq has several metaprogramming routes for deriving boilerplate. MetaCoq provides quoted syntax and verified metaprogramming; `coq-elpi` provides an embedded λProlog language integrated with Rocq elaboration and an existing derive framework. `rocq-json` uses `coq-elpi` because the task is structural inspection plus term construction. Compared with typical derivers for equality, ordering, showing, decidability, or logical instances, this tool generates executable code and a checked theorem relating two generated programs.

Production JSON libraries and language-specific deriving systems can generate efficient serializers outside Rocq. They do not, however, produce a Rocq-checked theorem tied to the source inductive or record declaration. The substantially new combination here is generated encoder, generated decoder, checked encode-decode theorem, extraction-conscious code shape, and a reproducible artifact that reports both successes and limitations.

6 Artifact

The artifact is part of the contribution. Tool papers benefit from evaluation that reviewers can rerun without reconstructing informal commands from prose. Our artifact provides a Docker path and a native path. The main script rebuilds the project from a clean state, runs the broad applicability benchmark, runs the extraction benchmark, and regenerates the LaTeX macros included in this paper.

The generated files are meant to be read directly. The JSON summary gives the aggregate result, the CSV files contain row-level data, and the Markdown failure report explains each failure category with representative Rocq diagnostics. This setup makes the boundary of the tool explicit: reviewers can see both what works and what is outside the supported fragment.

7 Limitations

The current deriver targets computational data in `Type` or `Set`. It does not attempt to serialize arbitrary propositions, dependent families, coinductive values, function fields, or records whose field types depend on earlier fields. Some nested recursion through standard containers is supported, but general nested and mutually recursive types remain outside the current design. This is a deliberate usability choice: partial support for a few accidental nested forms can be worse than a clear diagnostic when users cannot predict which structurally similar declarations will work.

The JSON representation is also intentionally modest. The paper evaluates the derivation method and the certified round-trip property, not complete adherence to the JSON RFC or interoperability with every external JSON parser.

8 Future Work

The main extensions are indexed families, proof-bearing data, and better library discovery. Indexed families may require fixed-index instances or dependent-pair decoding; proof fields likely require erasure plus user-supplied reconstruction lemmas. A Rocq-assisted discovery phase would also separate scanner limitations from deriver limitations. Finally, the declaration analysis used here could be factored into reusable ELPI infrastructure for other certified derivers that generate executable code and proofs together.

9 Conclusion

`rocq-json`'s ELPI deriver demonstrates that boundary code for extracted verified components can be treated as part of the formal artifact. The generated instances combine executable encoders and decoders with Rocq-checked invertibility proofs, apply across a meaningful fragment of existing library declarations, and extract to competitive code on the evaluated examples. The broader lesson is that certified derivation tools should be judged not only by the terms they generate, but also by the deployment boundary they secure and by the reproducibility of their claims about applicability and performance.

References

1. Lamiaux, T., Forster, Y., Sozeau, M., Tabareau, N.: Nested Inductive Types (2025), `https://hal.science/hal-05366368`, working paper or preprint
2. Tassi, E.: Elpi: rule-based meta-language for Rocq. In: CoqPL 2025 - The Eleventh International Workshop on Coq for Programming Languages. Denver, United States (Jan 2025), `https://inria.hal.science/hal-04990628`
3. The Rocq Development Team: The Rocq Prover. Zenodo, Version 9.0.0, `10.5281/zenodo.11551307` (Apr 2025). https://doi.org/10.5281/zenodo.15149629